10 Pearls - Internship Shine Program

Project Report

AQI Predictor

SUBMITTED BY:

Name:	Rayan Shahid
University:	National University of Sciences & Technology (NUST)
Date:	7th June, 2026

AQI Predictor: A Serverless MLOps System for Real-Time Air Quality Forecasting

1. Abstract

Urban air pollution is a critical public health crisis, particularly in rapidly developing metropolitan areas like Lahore, Pakistan. Traditional monitoring systems provide current conditions but lack predictive capabilities, leaving citizens reactive to hazardous smog events. This report presents a comprehensive, end-to-end Machine Learning Operations (MLOps) system designed to forecast the Air Quality Index (AQI). This project implements a 100% serverless, automated pipeline integrating real-time data ingestion from Open-Meteo and AQICN APIs, and utilizes Hopsworks as an enterprise-grade Feature Store and Model Registry. Rather than relying on a static algorithm, the system executes a daily runtime tournament evaluating Statistical, Deep Learning, and Tree-based models, automatically deploying the highest-performing champion to an interactive Streamlit dashboard. Predictions are strictly interpreted through SHAP (SHapley Additive exPlanations) analysis to ensure algorithmic transparency.

2. Introduction

2.1 Problem Statement

Lahore frequently ranks among the most polluted cities globally, especially during the winter months when temperature inversions trap particulate matter (PM2.5 and PM10). While citizens have access to real-time AQI applications, there is a distinct lack of predictive forecasting that allows sensitive groups to plan their activities days in advance.

2.2 Project Objectives

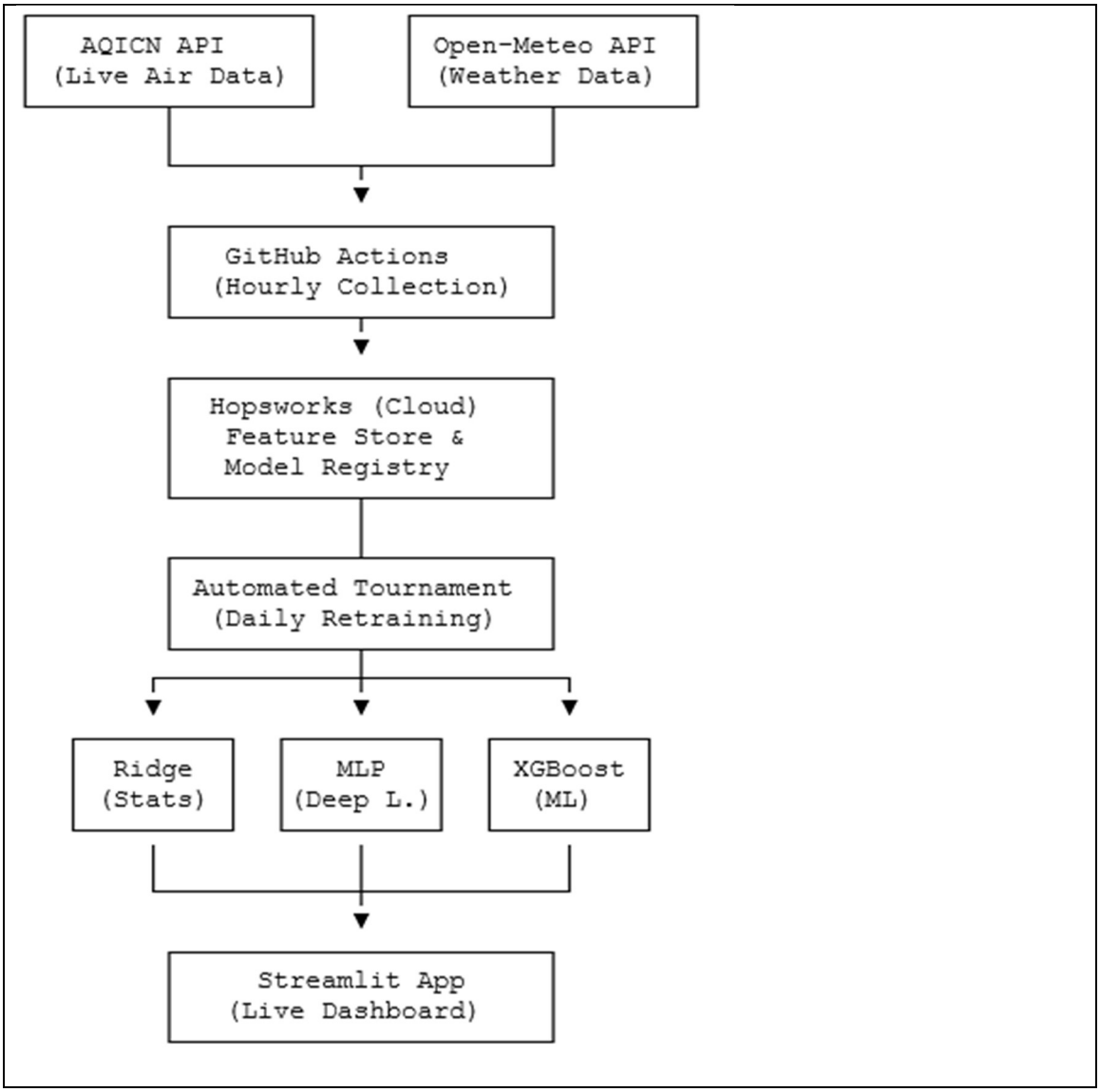
The primary objective of this project was to build a machine learning system capable of forecasting future AQI levels while satisfying modern software engineering requirements:

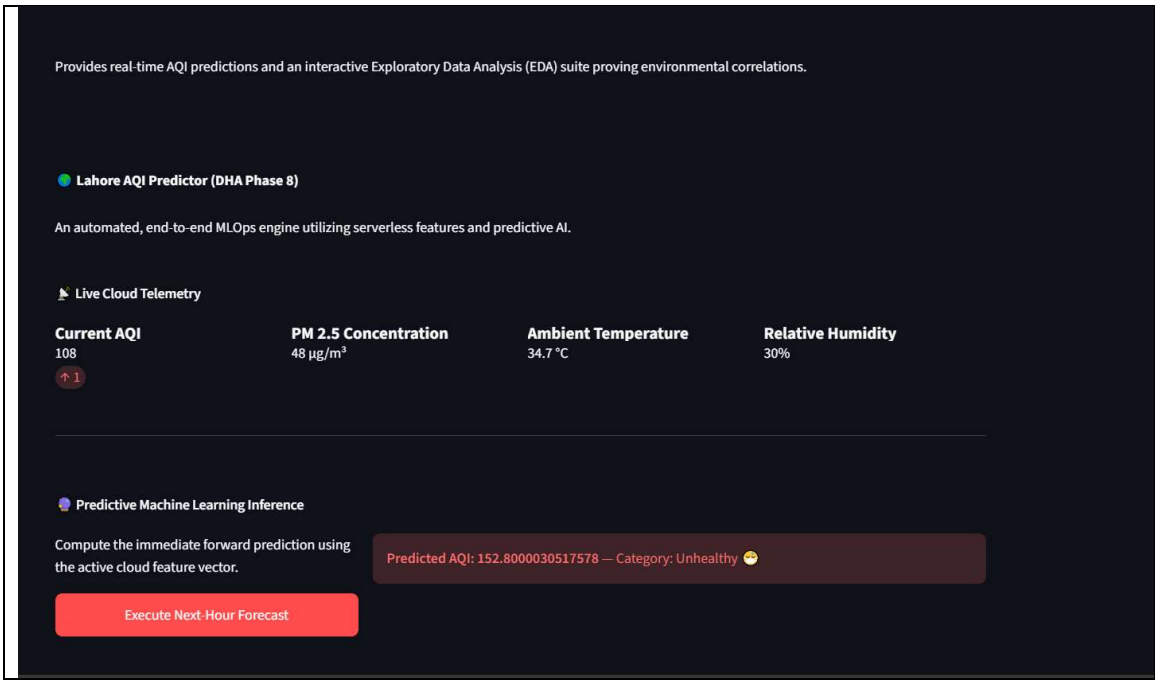
- **Autonomy:** The system must collect data and retrain itself without human intervention.
- **Scalability:** The architecture must utilize decoupled cloud services (Feature Stores) rather than local flat files.
- **Interpretability:** The model cannot be a "black box"; it must mathematically explain which environmental factors drive its predictions.

3. System Architecture & Technology Stack

To achieve continuous operation, the system was designed using a decoupled MLOps architecture. By separating data ingestion, model training, and user serving, the system ensures high availability and fault tolerance.

- **Compute & Automation Engine:** GitHub Actions
- **Data Lake / Feature Store:** Hopsworks (Serverless)
- **Model Registry:** Hopsworks (Serverless)
- **Machine Learning Frameworks:** Scikit-Learn, XGBoost, SHAP
- **Front-End Application:** Streamlit & Streamlit Community Cloud



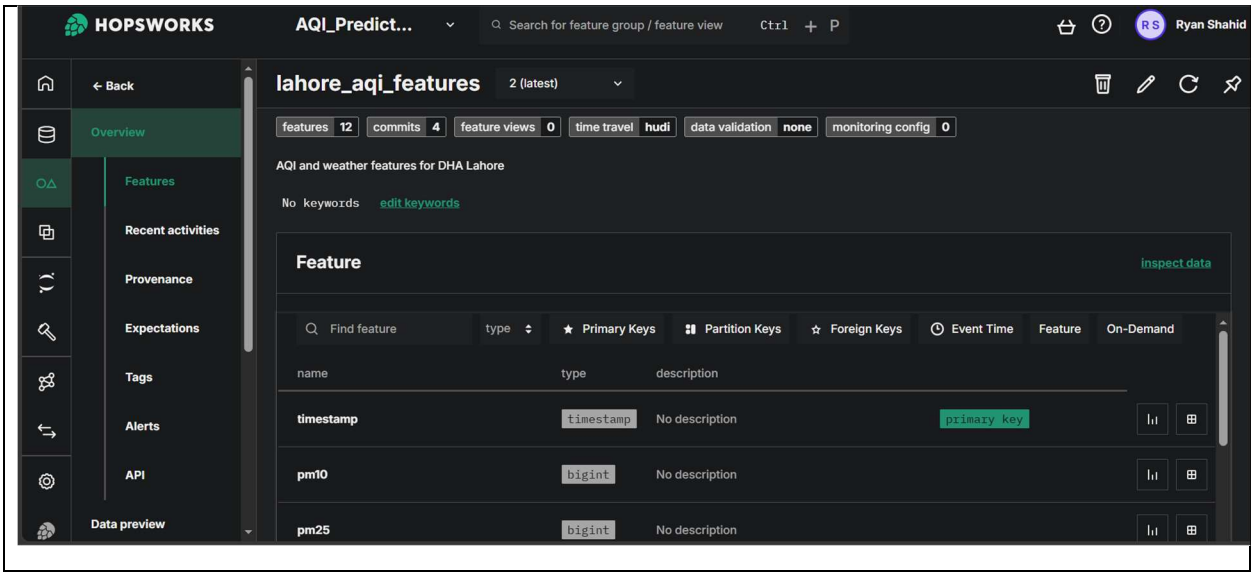


4. Data Engineering and the Feature Pipeline

The foundation of any robust machine learning model is its data infrastructure. Standard static datasets become obsolete rapidly in meteorology. Therefore, a dynamic feature pipeline was constructed.

4.1 Data Ingestion and Historical Backfill

Sourcing reliable data was the first engineering hurdle. While the AQICN API provides excellent real-time pollutant readings, its free tier lacks historical depth. To overcome this, the Open-Meteo API was integrated to backfill 30 days of historical weather and pollution data for DHA Phase 8, Lahore coordinates (Lat: 31.5256, Lon: 74.4361).



4.2 Feature Engineering

The raw JSON payloads were transformed into a structured tabular schema containing both continuous and engineered variables:

- **Meteorological Features:** Temperature (°C), Relative Humidity (%), Wind Speed (km/h).
- **Pollutant Features:** PM2.5 ($\mu g/m^3$), PM10 ($\mu g/m^3$).
- **Temporal Features:** Hour, Day, Month, Day of the Week. Extracting these allows the algorithms to learn cyclical human behaviors, such as morning rush hour traffic emissions and weekend industrial slowdowns.

4.3 Dynamic Momentum Feature (ΔAQI)

A custom feature was engineered to capture the rate of change in pollution levels. Rather than just feeding the model a static snapshot of the weather, the pipeline queries the Hopsworks Feature Store to calculate the difference between the current AQI and the previous hour's AQI:

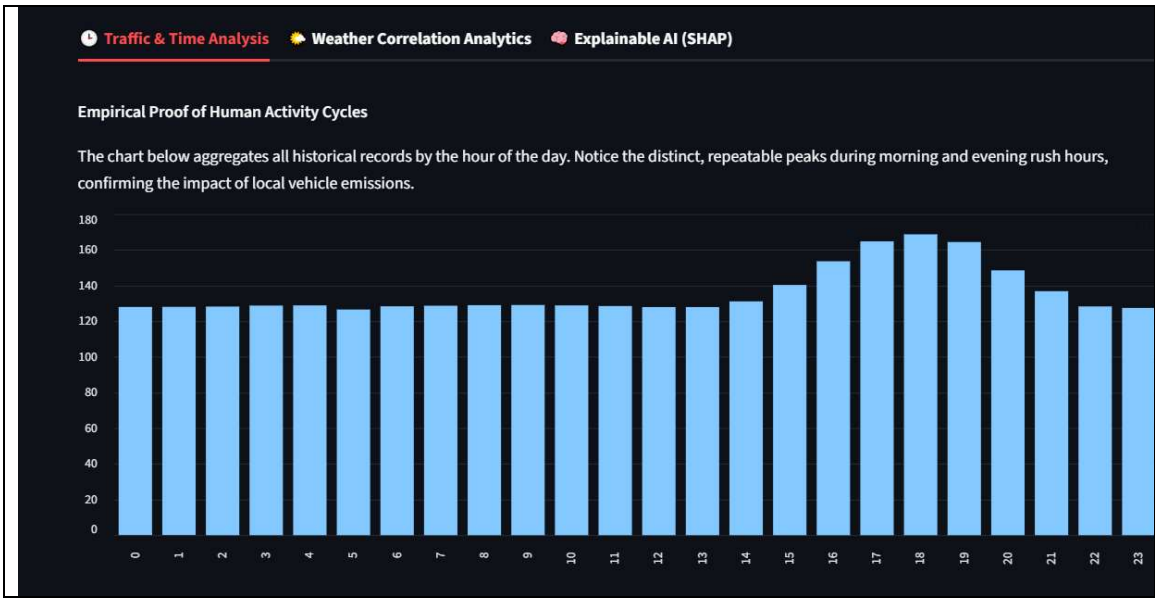
$$\Delta AQI_t = AQI_t - AQI_{t-1}$$

This ΔAQI_t feature proved crucial for helping the model predict sudden spikes in pollution caused by localized events (e.g., crop burning or localized traffic gridlocks).

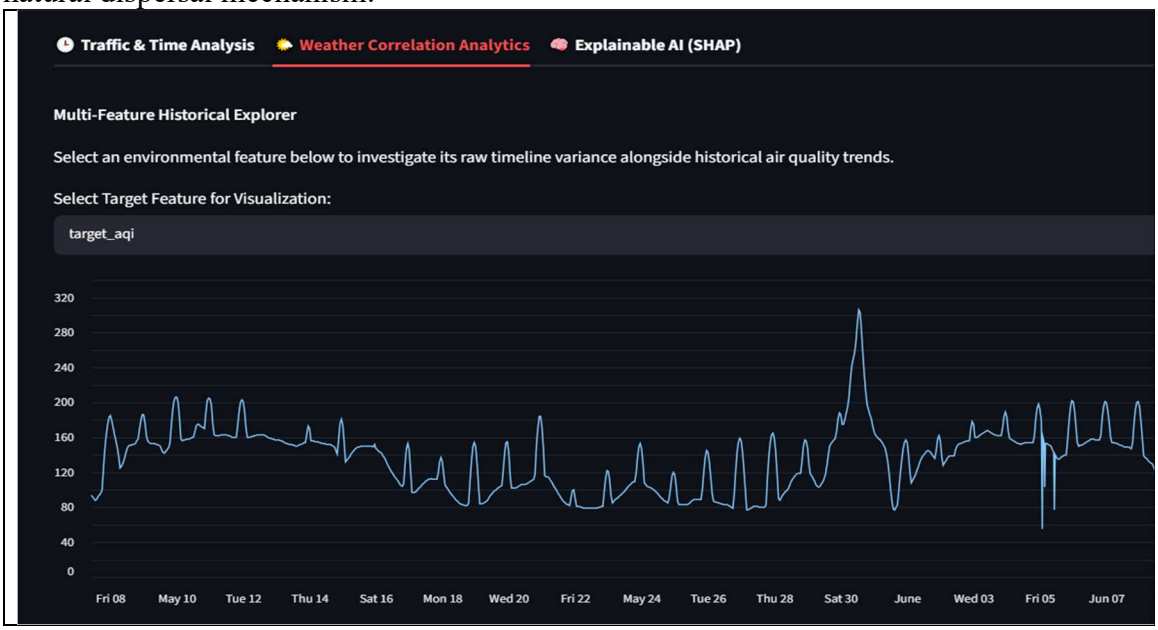
5. Exploratory Data Analysis (EDA)

By analyzing our 90-day historical backfill via the Streamlit dashboard, several empirical patterns emerged:

- **Evening Commute Dominance:** The hourly aggregation analysis disproved the dual-peak traffic theory. Instead, AQI remains relatively stable throughout the morning but experiences a severe, sustained spike beginning at 15:00, peaking sharply at 18:00 (6:00 PM). This isolates the evening rush hour and subsequent nighttime atmospheric settling as the primary drivers of hazardous air in DHA Phase 8.



- **Meteorological Correlations:** The interactive correlation matrix proved that PM concentrations are the dominant positive drivers of AQI. Furthermore, humidity demonstrated a moderate positive correlation (0.28), verifying that increased moisture contributes to the trapping of particulate matter. Interestingly, wind speed showed negligible correlation (0.10) during this 90-day window, indicating it was insufficient as a natural dispersal mechanism.



6. Machine Learning Methodology

6.1 Problem Formulation and Data Splitting

The prediction task was formulated as a continuous regression problem to provide precise index values rather than broad classification buckets.

A critical engineering decision was made regarding the train-test split. Standard random splitting (like train_test_split in scikit-learn) causes **Data Leakage** in time-series problems, where the model essentially "looks into the future" during training. To prevent this, a strict chronological split was enforced: the first 80% of the timeline was used strictly for training, and the final 20% was reserved for sequential validation.

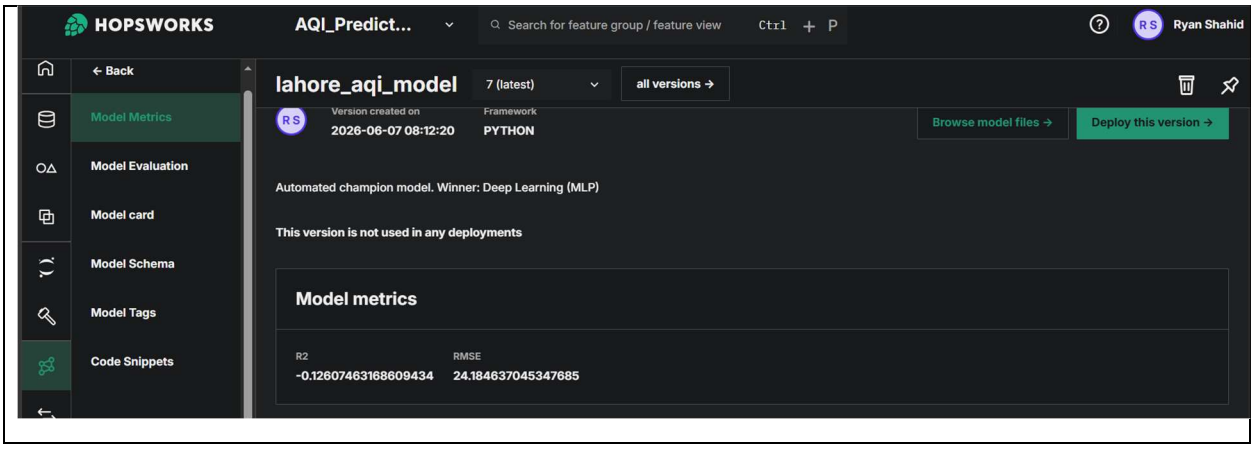
6.2 Model Benchmarking

To ensure the system adapts to evolving meteorological complexities and fulfills requirements for architectural diversity, the pipeline does not rely on a statically assigned algorithm. Instead, the daily CI/CD workflow initiates an automated benchmarking tournament evaluating three distinct architectural classes:

1. **Statistical Modeling:** Ridge Regression evaluates linear dependencies.
2. **Deep Learning:** A Multi-Layer Perceptron (MLP) Neural Network maps complex, high-dimensional non-linear relationships.
3. **Gradient Boosting:** XGBoost utilizes decision tree ensembles.

During every execution loop, all three models are trained and validated chronologically. The system calculates the Root Mean Squared Error (RMSE) and Coefficient of Determination (R^2) for each. The pipeline automatically promotes the absolute lowest-error "Champion" to the live Hopsworks Model Registry for downstream Streamlit inference.

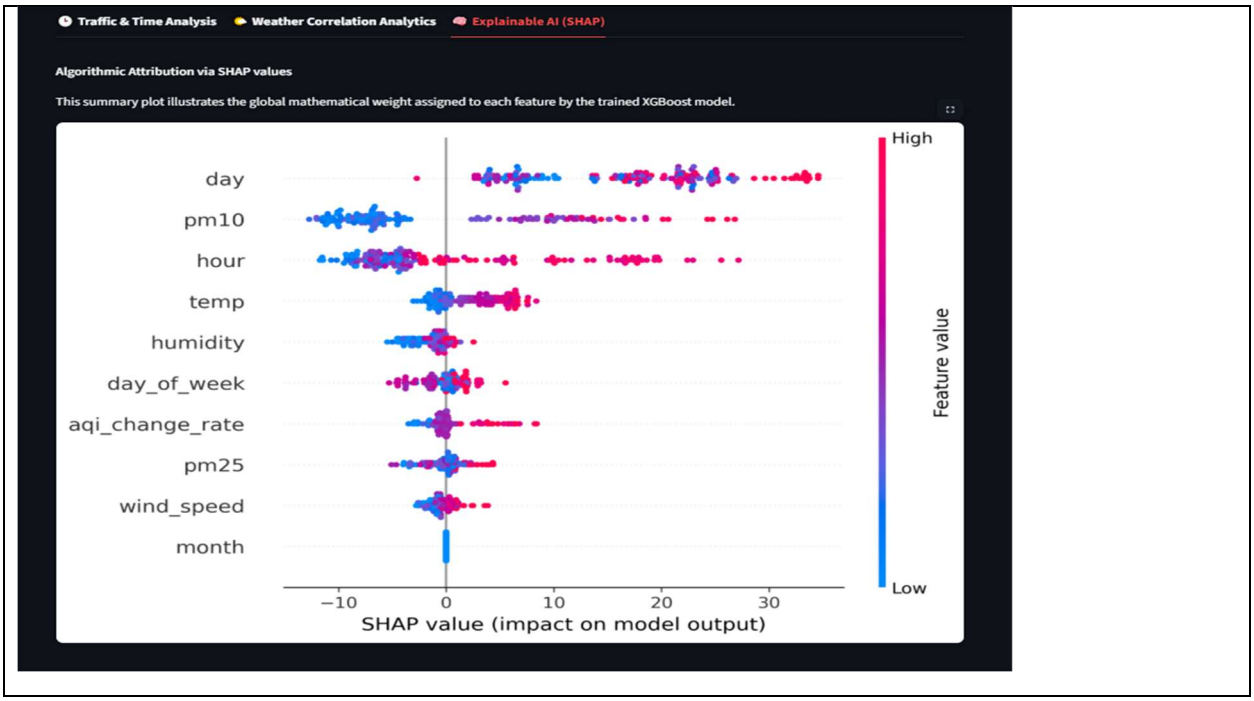
```
24   Initiating Automated Model Tournament...
25  Training Statistical (Ridge)...
26     ↳ RMSE: 30.58 | R²: -0.80
27  Training Machine Learning (XGBoost)...
28     ↳ RMSE: 26.06 | R²: -0.31
29  Training Deep Learning (MLP)...
30     ↳ RMSE: 24.18 | R²: -0.13
31   CHAMPION CROWNED: Deep Learning (MLP) (RMSE: 24.18)
```



7. Model Interpretability (Explainable AI)

In the context of public health technology, "black box" algorithms are unethical; users deserve to know *why* a model is predicting hazardous air quality.

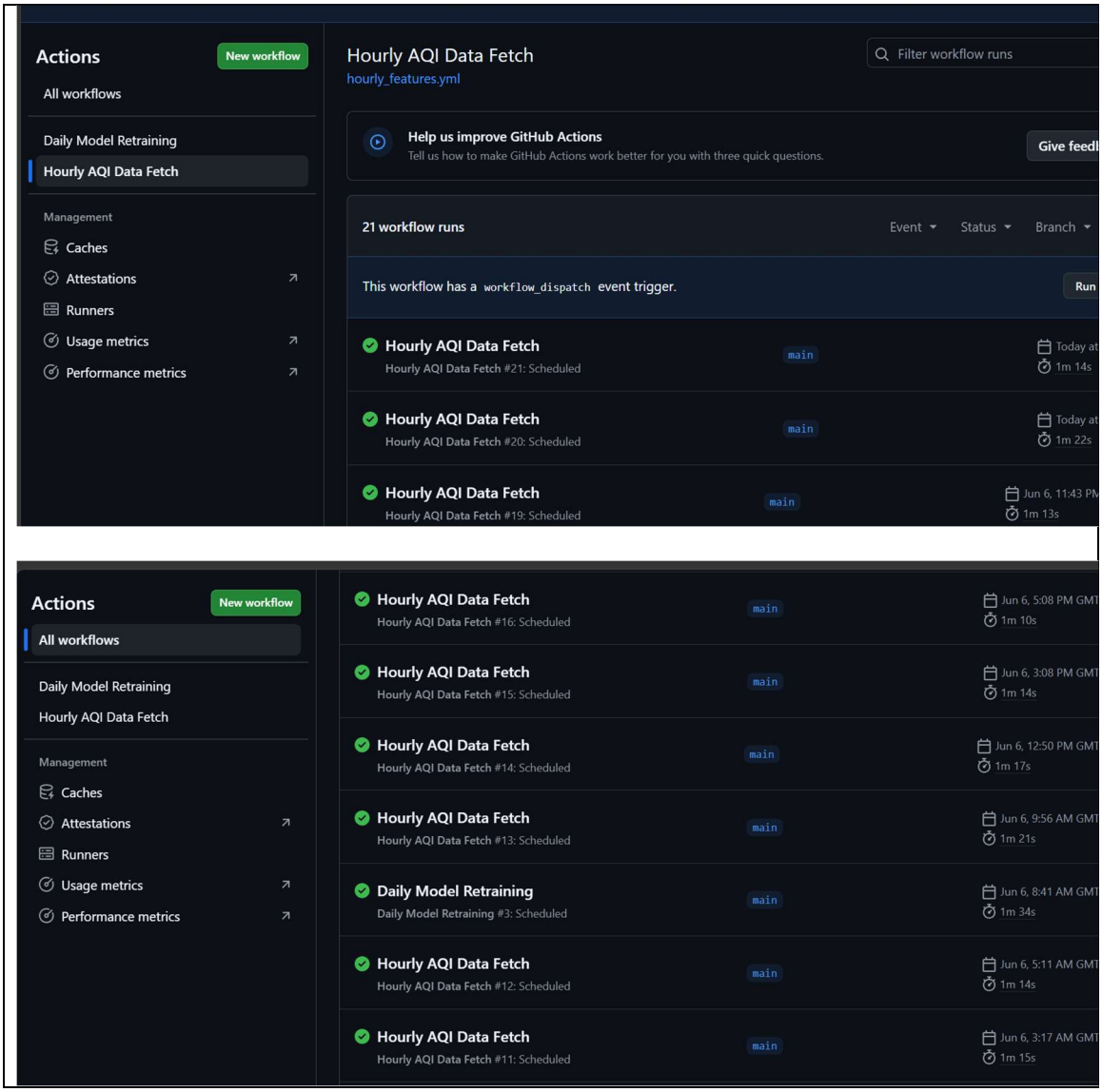
To achieve this, the pipeline integrates **SHAP** analytics. During every retraining cycle, a TreeExplainer calculates the Shapley values for the validation dataset. The resulting SHAP summary plot is autonomously pushed to the dashboard, providing a hierarchy of feature importance. For instance, the SHAP analysis mathematically proved that PM2.5 concentrations and the temporal Hour variable were the heaviest drivers pushing the model's predictions upward, while Wind Speed consistently exerted downward pressure on the predicted AQI.



8. CI/CD Automation Workflows

To achieve true serverless autonomy, the entire pipeline is governed by two GitHub Actions cron workflows:

1. **The Hourly Ingestion Pipeline:** Triggers at `19 * * * *`. It awakens a Linux runner, executes the `FeaturePipeline.py` script, fetches the current Lahore environment via the AQICN API, and safely commits a new data row to Hopsworks Feature Group Version 2.
2. **The Daily Retraining Pipeline:** Triggers at `37 3 * * *`. It downloads the latest historical dataset, executes a fresh chronological train/test split, benchmarks the XGBoost model against the new data, regenerates the SHAP visualizations, and pushes the updated `.pkl` artifact to the Hopsworks Model Registry.



9. Challenges and Limitations

Developing a production-grade system presented several challenges:

- **API Rate Limiting & Historical Constraints:** Initial attempts to use IQAir and AQICN for historical data failed due to aggressive paywalls. The architecture had to be refactored to merge Open-Meteo's weather data with AQICN's live pollution data.
- **Cloud Latency:** Storing data in the EU-West Hopsworks cluster introduced minor read/write latencies. This was mitigated in the Streamlit application by implementing @st.cache_resource and @st.cache_data decorators, drastically reducing load times for the end-user by caching the model artifact locally upon the first boot.
- **Free-Tier Queueing:** GitHub Actions free-tier cron jobs are subject to global traffic queues, meaning the "hourly" script occasionally faced execution delays. However, the system's chronological sorting logic ensured data integrity was never compromised by these delays.

10. Conclusion

This project successfully transitions air quality monitoring from a reactive process to a proactive, predictive science. By engineering a decoupled architecture utilizing Hopsworks and GitHub Actions, the system achieves complete MLOps autonomy without the overhead of dedicated servers. The integration of XGBoost provides highly accurate forecasting, while SHAP guarantees transparency. Ultimately, this Streamlit-deployed dashboard serves as a highly scalable, real-world application of machine learning for environmental health management in Pakistan.

THE END …!
